

Actividad #1 de cierre de brechas

Contexto del negocio

Una compañía de logística urbana opera una plataforma que coordina entregas realizadas por múltiples operadores de transporte en diferentes ciudades. Actualmente, cada operador envía información de sus entregas usando formatos distintos y con reglas particulares.

El sistema actual presenta varios problemas:

- Cada nuevo operador requiere modificaciones extensas sobre código existente.
- Existen validaciones repetidas en múltiples módulos.
- Los errores de procesamiento son difíciles de identificar y rastrear.
- El proceso de incorporación de nuevos formatos toma demasiado tiempo.
- Hay lógica mezclada entre validaciones, transformación de datos y persistencia.

La compañía desea construir un componente desacoplado y mantenible que permita centralizar el procesamiento de entregas provenientes de distintos operadores logísticos.

Objetivo

Desarrollar una API en .NET que permita recibir y procesar archivos de entregas de diferentes operadores logísticos, normalizando la información y generando un resultado consolidado del procesamiento.

Requerimiento funcional principal

La solución debe permitir:

1. Recibir archivos de entregas provenientes de múltiples operadores.
2. Procesar diferentes estructuras de archivo.
3. Validar reglas de negocio.
4. Consolidar los registros válidos.
5. Reportar errores encontrados durante el procesamiento.
6. Permitir incorporar nuevos formatos de operadores sin afectar los existentes.

Tipos de archivos iniciales

Operador A – CSV

OrderId, Customer, Address, DeliveryDate, Weight
ORD001, Carlos Ruiz, Calle 10 #20-30, 2026-05-01, 2.5
ORD002, Ana Torres, Carrera 15 #40-20, 2026-05-01, -1

Operador B – JSON

```
[  
{  
  "shipmentNumber": "SH900",  
  "recipient": "Laura Gómez",  
  "destination": "Av 30 #50-10",  
  "scheduledDate": "2026-05-01",  
  "packageWeight": 5  
}  
]
```

Operador C – TXT plano

PK100|Juan Perez|Calle 80 #10-50|20260501|3
PK101|Maria Diaz|Calle 90 #11-20|20260501|0

Reglas de negocio

- El identificador de la orden es obligatorio.
- El destinatario es obligatorio.
- La dirección de entrega es obligatoria.
- El peso del paquete debe ser mayor a 0.
- La fecha de entrega no puede ser inferior a la fecha actual.
- No se deben procesar órdenes duplicadas dentro del mismo archivo.

Resultado esperado del procesamiento

La API debe retornar:

- Total de registros procesados.
- Total de registros exitosos.
- Total de registros fallidos.
- Listado de errores encontrados.
- Registros normalizados válidos.

Requerimientos técnicos

La solución debe incluir:

- API REST en .NET
- Persistencia en memoria o base de datos ligera
- Manejo adecuado de excepciones
- Logging
- Validaciones desacopladas
- Arquitectura organizada y mantenible
- Pruebas unitarias

Escenario adicional

Después de finalizar el desarrollo inicial, la compañía solicita incorporar un nuevo operador llamado ExpressFast, cuyo formato es XML.

EXP001

Sebastian Mora

Calle 100 #20-10

2026-05-10

4

El sistema debe permitir incorporar este nuevo formato minimizando el impacto sobre funcionalidades existentes.

Entregables esperados

1. Código fuente completo.
2. Instrucciones de ejecución.
3. Colección Postman o Swagger funcional.
4. Pruebas unitarias.
5. Explicación breve de las decisiones técnicas tomadas.
6. Estrategia utilizada para soportar nuevos formatos futuros.

Aspectos que serán evaluados

- Diseño de la solución
- Organización del código
- Separación de responsabilidades
- Legibilidad y mantenibilidad
- Manejo de errores
- Escalabilidad de la solución
- Calidad de pruebas
- Capacidad de extensión
- Claridad técnica y criterio de implementación